| | |
|---|---|
| **Katedra Elektrotechniki Teoretycznej i Informatyki** | Zachodniopomorski Uniwersytet Technologiczny w Szczecinie |
| Przedmiot: Struktury danych i techniki programowania | Wydział Elektryczny |
| Numer ćwiczenia: 6 | Temat: Szablony klas. Implementacja własnych szablonów. |

**Szablony**

Język C++ jest językiem ścisłej kontroli typów. To znaczy, że pracując z kodem musimy mieć ciągle na uwadze ze zmiennymi jakich typów pracujemy. Wiele mechanizmów tego języka służy jednak zwiększeniu elastyczności języka:

- Niejawna konwersja typów,
- Cała idea klas, które można budować tak, aby symulowały typy wbudowane.

Problem jednak pojawia się, gdy chcemy napisać kod, który będzie działał na różnych typach danych:

- Funkcje – jeżeli parametrami funkcji będą zmienne określonych typów, to funkcja taka nie będzie dobrze działać dla zmiennych innych typów, lub próba jej wywołania spowoduje błąd kompilatora. Oczywiście funkcje można przeładowywać, ale to powodowałoby, że zmuszeni bylibyśmy pisać kilka wersji tej samej funkcji, różniących się tylko i wyłącznie typem parametrów.
- Klasy – jeżeli chcielibyśmy stworzyć klasę, w której główną funkcjonalnością byłoby przechowywanie tablicy i operacje na niej (tzw. kontener), to dużym ułatwieniem byłaby możliwość przechowywania w tablicy dowolnego typu zmiennych.

**Szablony funkcji**

Wyobraźmy sobie funkcję:

```
int Max(int a, int b)
{
	return a>=b?a:b;
}
```

Jej działanie dla argumentów innego typu jest inne:

```
double Max(double a, double b)
{
	return a>=b?a:b;
}
```

Można stworzyć dla tej funkcji tzw. Template (szablon), dzięki któremu raz zdefiniowana będzie mogła służyć do operacji na różnych typach zmiennych i zwracać wartości różnych typów.

```
template<typename TYP> TYP Max(TYP a, TYP b)
{
	return a>=b?a:b;
}
```

Część nagłówkowa: template (słowo kluczowe), <typename (słowo kluczowe) TYP – parametr szablonu, który lokalnie odwołuje się do typu zwracanego i argumentów>).
**Funkcja na podstawie swych argumentów „sama zgaduje", jaki typ danych ma być wstawiony w miejsce symbolicznej nazwy TYP.**

- Szablon funkcji reprezentuje zestaw (rodzinę) funkcji, działających dla dowolnej liczby typów danych.
- Szablon funkcji może być stosowany dla tych typów danych, dla których poprawne są wszystkie operacje, dokonywane na obiektach tychże typów w treści szablonu.

**Wywoływanie szablonu funkcji**

Szablony w C++ są implementowane w ten sposób, iż podczas kompilacji tworzony jest ich właściwy („nieszablonowy") kod dla każdego typu, dla którego używamy danego szablonu. Proces ten nazywamy **konkretyzacją** (ang. *instantiation*) a poszczególne egzemplarze szablonów – **specjalizacjami** (ang. *specialization* albo *instance*).

- „Normalne wywołanie", w naszym przypadku po prostu Max(a,b). Właściwy typ - parametr szablonu - może być wydedukowany z jej wywołania.
- Wywołanie z jawnym określeniem typu. W naszym przypadku np. Max<int>(a,b). W nawiasie ostrym podajemy parametry szablonu.

**Szablony klas**

**W C++ szablony klas możemy traktować jak** zestaw (rodzinę) klas, mogących przyjmować różne parametry lub np. współpracować z różnymi typami danych.

Ogólna składnia:

```
template <(opcjonalnie) typename TYP, typ NazwaParametru,…> class NazwaKlasy
{
	// ...
};
```

***Przykład 1.*** *Naszym celem będzie napisanie własnego kontenera (czyli klasy, która została stworzona do przechowywania jakiś danych, np. personaliów osób, danych liczbowych, czy innych) na bazie zwykłej tablicy statycznej. Pierwszym naszym celem będzie uczynienie tablicy bardziej elastycznej – niech szablon posłuży nam do tego, żeby liczba elementów tablicy była ustalana dopiero podczas tworzenia obiektu klasy.*

| Plik Tablica.h | Plik Tablica.cpp |
|---|---|
| `template<int NumberOfEl> class Tablica`<br>`/*w nawiasie <> mamy naszą zmienną NumberOfEl, która będzie sterować rozmiarem tablicy statycznej*/`<br>`{` | `#include "Tablica.h"`<br>`#include <iostream>`<br>`/*jest ważne, żeby pamiętać, że metody składowe szablonu są szablonami funkcji. Składnia ich definicji odpowiada zatem składni szablonów funkcji, z którymi zapoznaliście` |

```
	private:
	int Tab[NumberOfEl];
/*pole z tablicą statyczną, której liczba
elementów określona jest zmienną
NumberOfEl;*/

public:
	Tablica();//konstruktor
	void Set(int Element, int N);
	int Get(int N);
};
```

```
się wcześniej. Dodać należy jeszcze nazwę szablonu klasy
wraz z operatorem zasięgu*/

template<int NumberOfEl> Tablica<NumberOfEl>::Tablica()
{
	for (int i = 0; i < NumberOfEl; i++)
		Tab[i] = 0;
}
template<int NumberOfEl> void Tablica<NumberOfEl>::
Set(int Element, int N)
{
		Tab[N] = Element;
}
template<int NumberOfEl> int Tablica<NumberOfEl>::
Get(int N)
{
		return Tab[N];
}
```

Plik main.cpp

```
#include "Tablica.h"
#include "Tablica.cpp"
int main()
{
/*Tworzymy obiekt klasy Tablica, określając w tym momencie ile elementów będzie miał nasz kontener.
Szablon Tablica przekształca się w konkretną klasę, której pole jest tablicą 10 elementową*/
	Tablica<10> T;
/*Dla obiektu T wywołujemy funkcję Set ustawiając wartość elementu 9 na 20;*/
	T.Set(20, 9);
/*Wyświetlamy wartości elementów używając funkcji Get*/
	std::cout << T.Get(0) <<" "<< T.Get(9);
	std::cin.get();
}
```

***Przykład 2.*** *Nasz kontener z poprzedniego przykładu przyjmuje tylko liczby całkowite jako elementy. Szablonu możemy użyć także po to, żeby nasz kontener przyjmował różne typy danych. Zarówno te wbudowane, jak i pisane przez nas w postaci własnych klas i struktur.*

Plik Tablica.h

```
template<int NumberOfEl, typename Typ>
class Tablica
//tym razem oprócz wymiaru tablicy
//parametrem szablonu jest typ elementów
//tablicy
{
	private:
//wszędzie tam, gdzie wykorzystujemy
//parametryczny typ elementów musimy
//napisać to jawnie:
	Typ Tab[NumberOfEl];
		//unsigned Size;
	public:
		Tablica() {}//konstruktor
//należy pamiętać, że teraz funkcje Set i
//Get wykorzystują parametryczny typ
	void Set(Typ Element, int N);
	Typ Get(int N);
};
```

Plik Tablica.cpp

```
#include "Tablica.h"
#include <iostream>

template<int NumberOfEl, typename Typ> void
Tablica<NumberOfEl,Typ>:: Set(Typ Element, int N)
{
		Tab[N] = Element;
}
template<int NumberOfEl, typename Typ> Typ
Tablica<NumberOfEl,Typ>:: Get(int N)
{
		return Tab[N];
}
```

```
/*stwórzmy sobie bardzo prostą klasę
Example, żeby pokazać, że obiekty tej
klasy mogą być teraz elementami naszej
tablicy*/

class Example
{
public: int A;
};
```

Plik main.cpp

```
#include "Tablica.h"
#include "Tablica.cpp"

int main()
{
Example Obj;
	Obj.A = 10;
	//tutaj wskazujemy, że obiekt T będzie miał pole Tab będące statyczną tablicą 10 elementową,
której elementy to obiekty klasy Example
	Tablica<10,Example> T;
	T.Set(Obj, 9);
	//tym razem wyświetlamy wartość pola A elementu 9 Tablicy Tab
	std::cout << T.Get(9).A;
	std::cin.get();
}
```

**Zadania:**

1. **Zmodyfikuj szablon z przykładu 1 w ten sposób, że:**
   - **Konstruktor nadaje elementom tablicy z pola Tab wartości losowe z zakresu od 1 do 100**
   - **Funkcje Get i Set zastąpione są odpowiednim przeciążeniem operatora []. Dostęp do obiektu powinien być realizowany w dokładnie taki sam sposób, jak do elementu zwykłej tablicy, tzn. zadziałać ma poniższy fragment kodu:**
     - `Tablica<10> T;`
     - `cout << T[9];`
     - `cin.get();`
   - **Dodaj metody: Maksimum (wyszukującą maksimum w tablicy), Minimum (wyszukującą minimum w tablicy) , Sortuj (sortującą tablicę wybraną metodą), Wyświetl (wyświetlającą całą tablicę).**
2. **Na podstawie powyższego zadania zbuduj kolejny szablon, tym razem z polem Tab2D będącym tablicą dwuwymiarową.**
3. **Stwórz klasę Osoba z polami Imię, Nazwisko (typu string) oraz odpowiednim konstruktorem. Przeciąż w klasie operatory << oraz < i > (operatory < > mają porównywać dwa obiekty pod względem kolejności alfabetycznej ich pól Nazwisko). Zmodyfikuj szablon z przykładu 2 uzupełniając go tak, jak w zadaniu 1 (z wyjątkiem konstruktora). Napisz program realizujący następujące zadania:**
   - **utworzenie 5 obiektów klasy Osoba,**
   - **utworzenie obiektu klasy Tablica,**
   - **przypisanie obiektów klasy Osoba do elementów kontenera Tablica,**

- **wyświetlenie zawartości kontenera,**
- **posortowanie zawartości kontenera,**
- **wyświetlenie posortowanej zawartości**